

ANIMUS

Canonical Architecture v2.0

Reconciled system design and 10/10 improvement standard

Purpose

This document reconciles the Mind Vision, Context Spine, Memory Architecture, Intelligence Swarm, Architecture Book v1, True System Design Book v1, and the modular v1 system-design documents into one authoritative architecture. It also converts every known weakness into a defined target state, engineering remedy, acceptance test, and roadmap obligation.

Status: CANONICAL DESIGN BASELINE

Version 2.0 | June 17, 2026 | Arete / Animus Project

Document Control

Field	Value
Authority	Canonical architecture and governing vocabulary for Animus.
Supersedes	Animus Architecture Book v1.0 and Animus True System Design Book v1.0 as architectural authorities.
Retains	Mind Vision as philosophical charter; original files as provenance and design history.
Supporting specifications	Context Spine, Memory, Agent Runtime, MCP, Data/Retrieval/Graph, Intelligence/Forecasting, Governance/Security/Evaluation/Operations.
Change rule	Any contradictory implementation or document must be updated, explicitly excepted by an Architecture Decision Record, or deprecated.
Completion rule	No subsystem is production-ready until it has schema, examples, tests, logs, ownership, failure behavior, security controls, and measurable exit criteria.

Executive Determination

The source set contains a strong and coherent vision, but it mixes conceptual faculties, architecture layers, product modules, storage categories, and folder domains as though they were equivalent. v2 resolves this by assigning one purpose to each model and establishing a single chain of authority from philosophy to implementation.

The reconciliation rule

Mind faculties describe what Animus must be capable of. Technical planes describe how those capabilities are implemented and governed. Context Spine domains describe where canonical information belongs. Runtime modules describe deployable software. Artifact types describe the evidence the system creates. None of these models may substitute for another.

The 10/10 Standard

A score of 10/10 does not mean perfection or completion. It means the architecture is complete enough to implement, test, operate, recover, audit, and evolve without relying on hidden assumptions.

Dimension	10/10 condition
Clarity	One authoritative term for each concept; no competing layer models.
Completeness	Architecture covers data, runtime, security, operations, evaluation, failure, ownership, and lifecycle.
Implementability	Every subsystem has contracts, schemas, state transitions, and dependency boundaries.

Dimension	10/10 condition
Testability	Requirements map to automated or reviewable acceptance tests.
Traceability	Outputs preserve provenance, transformations, decisions, tool use, and version lineage.
Safety	Least privilege, approval gates, threat controls, rollback, and deletion/export are specified.
Operability	Deployment, observability, backup, recovery, cost, and incident handling are defined.
Adaptability	Versioning, migrations, model/provider portability, and ADR governance are built in.
Cognitive quality	Memory, retrieval, reasoning, forecasting, and reflection improve through measured outcomes.
Human sovereignty	The user retains final authority, visibility, correction, veto, and control of personal information.

Contents

1. 1. Canonical Definition and Philosophy
2. 2. Reconciliation of the Existing Source Set
3. 3. Canonical Architecture Model
4. 4. Context Spine and Context Envelope
5. 5. Canonical Object and Event Ontology
6. 6. Memory Architecture
7. 7. Agent Runtime and Swarm Coordination
8. 8. MCP Tooling and Permission Architecture
9. 9. Data, Retrieval, and Knowledge Graph
10. 10. Intelligence, Pattern Recognition, and Forecasting
11. 11. Governance, Security, and Threat Model
12. 12. Evaluation, Observability, and Learning
13. 13. Deployment, Operations, and Recovery
14. 14. Roadmap, Phase Gates, and Acceptance Criteria
15. 15. 10/10 Architecture Audit and Gap Closure
16. 16. Architecture Decision Records
17. 17. Source Migration and Document Authority
18. Appendix A. Canonical Schemas
19. Appendix B. State Machines and Control Flows
20. Appendix C. Initial Backlog

1. Canonical Definition and Philosophy

Canonical definition

Animus is a governed, persistent personal intelligence system that builds an evidence-linked model of the user, their projects, and the external world; retrieves relevant context; coordinates specialized reasoning and tools; produces calibrated assessments and forecasts; learns from outcomes; and acts only within explicit human authority.

1.1 Purpose

- **Augment judgment, not replace it.** Animus should widen perception, preserve context, challenge assumptions, expose uncertainty, and improve execution.
- **Create continuity.** The system should carry goals, decisions, lessons, and unresolved questions across sessions, tools, projects, and years.
- **Convert information into intelligence.** Raw data becomes observations, signals, patterns, hypotheses, assessments, forecasts, decisions, actions, and outcome reviews.
- **Make competence inspectable.** Important outputs must show evidence, assumptions, confidence, alternatives, and the path by which they were produced.
- **Protect sovereignty.** The user owns the identity model, memory, permissions, corrections, exports, and final decisions.

1.2 Foundational principles

Principle	Architectural consequence
Human sovereignty	No silent high-impact action; correction, veto, export, and deletion are first-class capabilities.
Skeptical trust	Trust derives from provenance, repeatability, calibration, outcome review, and transparent limitations.
Truth seeking	Facts, claims, interpretations, hypotheses, forecasts, and advice remain visibly distinct.
Information sovereignty	Private context is portable, vendor-independent where practical, and protected by policy and technical controls.
Long-horizon cognition	The system preserves temporal history, decisions, forecast outcomes, and longitudinal patterns.
Arete	The system is optimized for excellence through disciplined feedback, not comfort through agreement.
Plural reasoning	Competing hypotheses and minority interpretations are preserved until evidence resolves them.
Bounded agency	Tools and agents operate through explicit contracts, budgets, permissions, and reversible workflows.

2. Reconciliation of the Existing Source Set

The existing documents are not discarded. Each is assigned a stable role in the design lineage. Content is either retained, expanded, relocated, or superseded.

Source	Disposition	Canonical role
Animus Mind Vision	Retain	Philosophical charter and design intent. It does not define technical boundaries.
Animus Context Spine	Integrate and expand	Origin of the root domains. Replaced technically by the v2 domain model, context envelope, ownership, and lifecycle rules.
Animus Memory Architecture	Integrate and expand	Five memory classes retained; now governed by object ontology, temporal validity, contradiction handling, consolidation, and write policy.
Animus Intelligence Swarm	Integrate and expand	Agent intent retained; replaced operationally by runtime contracts, orchestration, budgets, disagreement handling, and evaluation.
Animus Architecture Book v1	Supersede	Useful summary and historical snapshot; too broad to be an implementation authority.
True System Design Book v1	Supersede as master	Strong baseline; v2 replaces its six-layer model with the canonical faculties, planes, modules, artifacts, and domain structure.
Modular v1 documents 01-08	Retain as source specifications	Their content is absorbed here and should be revised to conform to v2 terminology and contracts.

2.1 Conflicts resolved

Conflict	Resolution
Six core layers vs interface/cognitive/memory/core vs Core/Forge/Swarm	None are the canonical architecture model. Faculties are conceptual; planes are technical; modules are deployable.
FORECASTS separate from INTELLIGENCE	FORECAST is an artifact type inside the intelligence lifecycle. The FORECASTS domain remains a curated view and review workspace, not a separate truth store.
MEMORY separate from KNOWLEDGE	Memory records relevance to the Mind and its history. Knowledge records claims about subjects. A single object may have links to both but has one canonical owner.
Agent outputs written directly to memory	Agents write proposed artifacts to a staging ledger. Promotion into canonical memory follows validation and policy.
Storage technologies presented as architecture	Technologies are implementation choices behind logical stores and may change through ADRs.
Confidence used as one generic score	v2 separates source reliability, claim confidence, model confidence, retrieval relevance, and forecast probability.
Human approval stated but not executable	v2 defines risk classes, approval receipts, scopes, expiration, idempotency, and rollback.

3. Canonical Architecture Model

3.1 Mind faculties: what Animus does

Faculty	Responsibility	Primary evidence
Identity	Represents goals, values, roles, constraints, preferences, and operating principles.	Versioned identity and policy objects.
Perception	Collects and normalizes user input, files, systems, APIs, and public data.	Source, observation, and ingestion records.
Memory	Preserves events, decisions, lessons, preferences, contradictions, and continuity.	Temporal memory objects and consolidation logs.
Understanding	Resolves entities, relationships, claims, context, and relevance.	Knowledge graph, retrieval traces, claim records.
Reasoning	Forms hypotheses, compares alternatives, plans, critiques, and synthesizes.	Reasoning artifacts, assumptions, alternatives, evidence maps.
Imagination	Builds scenarios, forecasts, counterfactuals, and second-order effects.	Forecasts, scenario trees, indicators, outcome reviews.
Agency	Executes bounded actions through tools and workflows.	Tool calls, approvals, action receipts, rollback records.
Reflection	Reviews outcomes, calibrates models, extracts lessons, and updates strategy.	Evaluations, retrospectives, memory promotions, calibration reports.

3.2 Technical planes: how Animus runs

Plane	Purpose	Key services
Experience plane	Human interaction and capture.	Chat, voice, dashboard, files, notifications, mobile capture.
Control plane	Policy, identity, orchestration, permissions, budgets, approvals.	Policy engine, workflow controller, agent registry, approval service.
Cognitive plane	Retrieval, model routing, reasoning, synthesis, simulation.	Context builder, router, planners, critics, forecast engine.
Data plane	Canonical objects, events, documents, graph, vector and analytical stores.	Object registry, event ledger, document store, graph, warehouse.
Tool plane	MCP services and external integrations.	Filesystem, memory, web, government, market, calendar, email, code, projects.
Assurance plane	Security, observability, evaluation, provenance, recovery.	Audit log, traces, evals, incident controls, backup, restore.

3.3 Deployable runtime modules

- **Context Service:** builds the bounded context envelope for each task.
- **Object Registry and Event Ledger:** canonical identities, versions, state transitions, and provenance.
- **Memory Service:** capture, candidate writes, validation, consolidation, recall, contradiction, and deletion.

- Retrieval Service: hybrid retrieval, reranking, source balancing, deduplication, and explanation.
- Agent Runtime: contracts, execution sandboxes, budgets, checkpoints, and output validation.
- Workflow Orchestrator: deterministic and agentic steps, retries, approvals, compensation, and resume.
- MCP Gateway: tool discovery, permission enforcement, request signing, logging, and rate control.
- Intelligence Engine: observation-to-forecast lifecycle, pattern library, watch indicators, and reviews.
- Evaluation Service: offline suites, online quality signals, calibration, and regression gates.
- Operations Service: deployment, health, cost, backup, restore, incident, and change management.

3.4 End-to-end cognitive loop

Capture -> Normalize -> Resolve -> Store evidence -> Build context -> Reason
 -> Compare alternatives -> Request approval when required -> Act
 -> Observe outcome -> Evaluate -> Consolidate memory -> Update models

Non-negotiable invariant

No final answer, forecast, memory write, or external action may lose the evidence chain that explains where it came from, what transformed it, and under which policy it was accepted.

4. Context Spine and Context Envelope

4.1 Canonical domains

Domain	Owns	Does not own
SELF	Identity, values, roles, goals, constraints, preferences, personal policies.	Raw conversations, external facts, general reference material.
WORLD	Time-sensitive external state and observed events.	Stable reference knowledge or interpretations.
PROJECTS	Project objectives, requirements, decisions, plans, dependencies, artifacts.	General agent definitions or global SOPs.
INTELLIGENCE	Signals, patterns, hypotheses, assessments, scenarios, forecasts, indicators, reviews.	Raw source documents.
MEMORY	Experience-linked events, decisions, lessons, preferences, unresolved questions, meta-memory.	Every fact the system has ever seen.
KNOWLEDGE	Entities, concepts, claims, documents, ontologies, methods, stable references.	Personal salience or autobiographical history unless linked.
OPERATIONS	Agents, MCPs, workflows, schemas, evals, runbooks, policies, deployments.	Business or personal project content.
VIEWS	Daily briefs, forecast watchlists, dashboards, reports, curated collections.	Canonical truth; views are generated from source objects.
ARCHIVE	Immutable snapshots, deprecated versions, source captures, exports, retired projects.	Active canonical state.

4.2 Why FORECASTS becomes VIEWS

Forecast is an artifact type in the intelligence lifecycle, not a separate ontology. A dedicated forecast workspace remains useful, but it should be a generated view over canonical forecast objects. This eliminates duplicate truth and keeps forecasts linked to their signals, hypotheses, decisions, and outcomes.

4.3 Canonical context envelope

```

context_id: string
actor: {user_id, agent_id, role}
objective: {task, success_criteria, deadline}
scope: {domains, projects, entities, time_horizon}
state: {current_state, open_questions, prior_decisions}
constraints: {policy, privacy, safety, cost, latency}
evidence: [{object_id, type, relevance, reliability, excerpt_hash}]
contradictions: [{claim_a, claim_b, resolution_status}]
tools: [{tool_id, permissions, risk, budget}]
output_contract: {artifact_type, schema_version, validation}
trace: {request_id, parent_trace_id, model, retrieval_run_id}
uncertainty: {known_gaps, assumptions, confidence_dimensions}

```

4.4 Context assembly policy

1. Classify the task by domain, sensitivity, horizon, action risk, and output contract.
2. Retrieve from canonical objects before generated views or summaries.
3. Use semantic, lexical, metadata, graph, and temporal retrieval as independent candidate channels.
4. Rerank with relevance, source reliability, temporal validity, importance, diversity, and contradiction coverage.
5. Allocate a context budget by evidence type; do not let one source or memory class monopolize the window.
6. Attach source IDs and transformation lineage to every context segment.
7. Record omitted but material contradictions and unresolved gaps.
8. Persist a retrieval trace sufficient to reproduce or diagnose the context selection.

5. Canonical Object and Event Ontology

The v1 metadata object is useful but too generic. v2 separates universal envelope fields from type-specific payloads and makes time, provenance, state, and versioning explicit.

5.1 Universal object envelope

Field group	Required content
Identity	object_id, object_type, schema_version, canonical_domain, owner, tenant.
Version	version_id, supersedes, created_at, updated_at, valid_from, valid_to.
Lifecycle	state: candidate, active, validated, disputed, superseded, deprecated, archived, deleted.
Provenance	source IDs, acquisition method, transformations, producing agent/workflow/model, hashes.
Quality	source reliability, claim confidence, freshness, completeness, review status.
Security	classification, sensitivity, access policy, retention policy, encryption scope.
Relationships	typed edges, entity references, parent/child lineage, project and context links.

Field group	Required content
Content	type-specific payload plus human-readable summary.

5.2 Core artifact types

Artifact	Definition	Required distinction
Source	Original evidence object or immutable capture.	Not an interpretation.
Observation	Atomic extraction from a source.	Describes what was seen, not what it means.
Claim	Proposition attributable to a source or actor.	May be true, false, disputed, or unverified.
Entity	Resolved person, organization, project, place, system, concept, or instrument.	Identity resolution is versioned and reversible.
Event	Time-bounded occurrence involving entities and state change.	Uses event time and system time.
Signal	Observation or metric deviation considered potentially meaningful.	Contains baseline and detection method.
Pattern	Repeated or cross-domain structure supported by multiple signals.	Contains scope, recurrence, and counterexamples.
Hypothesis	Testable explanation for a pattern or outcome.	Contains falsifiers and competing hypotheses.
Assessment	Evidence-weighted judgment at a stated time.	Contains evidence map and confidence dimensions.
Forecast	Time-bound probabilistic proposition or scenario.	Contains probability, resolution rule, review date, indicators.
Decision	Choice among alternatives under constraints.	Contains rationale, authority, assumptions, and revisit trigger.
Action	Executed or proposed operation.	Contains approval, tool receipt, effects, rollback.
Outcome	Observed result connected to forecast, decision, or action.	Contains measurement and attribution limits.
Lesson	Generalization derived from outcomes.	Contains applicability, exceptions, and evidence.

5.3 Bitemporal truth

Important objects store both valid time (when the fact or state was true in the world) and system time (when Animus learned or changed its representation). This permits historical reconstruction, correction without erasure, and audit of what the system knew when a decision was made.

6. Memory Architecture

6.1 Memory classes retained and clarified

Class	Canonical purpose	Write rule
Working	Bounded state for the active task or workflow.	Ephemeral by default; checkpoint only when needed for resume or audit.
Episodic	Events, interactions, milestones, decisions, and sequences tied to time.	Candidate write, deduplicate, identify actors, preserve source.
Semantic	Stable claims, concepts, procedures, and resolved knowledge.	Requires evidence, validity scope, and contradiction check.
Reflective	Lessons, strategies, behavioral patterns, and generalized learning.	Requires multiple evidence points or explicit human confirmation.
Meta	Confidence, contradictions, gaps, decay, source quality, retrieval and memory performance.	System-generated but reviewable; never treated as ground truth without context.

6.2 Memory write pipeline

Propose -> classify sensitivity -> normalize -> resolve entities -> detect duplicates
 -> compare contradictions -> score salience -> select memory class
 -> validate policy -> human review when required -> commit version
 -> index -> schedule review/decay -> record write receipt

6.3 Salience and retention

Factor	Question
Future utility	Is this likely to improve a later decision, project, relationship, or workflow?
Uniqueness	Does it add information rather than repeat an existing memory?
Stability	Is it persistent, temporary, or likely to change?
Consequence	Would forgetting it create material cost, risk, or confusion?
User intent	Did the user explicitly ask to remember or forget it?
Sensitivity	Would storing it create privacy, dignity, or security risk?
Evidence quality	Can it be traced and reviewed?

6.4 Contradiction model

- Never silently overwrite a contradictory memory.
- Create a contradiction object linking the competing claims, sources, valid times, and confidence dimensions.
- Distinguish change over time from factual disagreement.
- Allow statuses: unresolved, contextualized, corrected, superseded, or intentionally plural.
- Use the contradiction registry during retrieval and expose material conflicts to reasoning agents.

6.5 Consolidation and forgetting

- Episodic memories may produce summaries, but originals remain recoverable until retention policy permits deletion.
- Reflective lessons are periodically tested against newer outcomes and may be narrowed or revoked.
- Stale semantic claims receive review dates and freshness decay rather than automatic deletion.
- User-directed forgetting creates a deletion tombstone and cascades to indexes, derived views, and exports where technically possible.
- Memory quality is measured by precision, recall, contradiction visibility, write correctness, and later utility.

7. Agent Runtime and Swarm Coordination

7.1 Agent design rule

Agents are bounded workers, not personalities

An agent exists only when a recurring task benefits from a stable contract, restricted tools, specific output schema, measurable quality, and independent evaluation. Otherwise it is a prompt or workflow step, not an architectural component.

7.2 Canonical agent contract

Contract section	Required fields
Identity	agent_id, version, owner, mission, non-goals.
Invocation	allowed triggers, preconditions, priority, concurrency.
Inputs	schema, permitted domains, freshness and evidence requirements.
Outputs	artifact types, schema versions, evidence and uncertainty requirements.
Tools	allowlist, risk level, operation limits, credentials, network scope.
Memory	read filters, write proposal types, forbidden data, retention.
Budgets	time, tokens, calls, cost, data volume, recursion depth.
Control	checkpointing, cancellation, retries, idempotency, escalation.
Quality	validators, eval suite, minimum thresholds, abstention conditions.
Failure	known failure modes, safe state, dead-letter behavior, operator action.

7.3 Swarm role catalog

Role	Canonical responsibility	Boundary
Collectors	Acquire sources and immutable captures.	Do not interpret strategic meaning.
Normalizers	Convert sources into canonical objects.	Do not promote uncertain claims as facts.

Role	Canonical responsibility	Boundary
Resolvers	Resolve entities, time, duplicates, and relationships.	Must preserve ambiguity and alternatives.
Researcher	Extract claims, methods, evidence, and gaps.	Does not issue final strategic judgment.
Pattern Analyst	Compare signals across time and domains.	Must show baselines, counterexamples, and support.
Forecaster	Generate probabilistic forecasts and scenarios.	Cannot convert confidence into false certainty.
Critic / Red Team	Search for disconfirming evidence and failure modes.	Cannot silently rewrite primary analysis.
Oracle / Synthesizer	Integrate competing analyses into decision support.	Must preserve dissent and source trace.
Memory Curator	Review candidate memory, consolidation, and contradiction.	Does not decide sensitive writes outside policy.
Project Operator	Maintain plans, dependencies, tasks, and retrospectives.	External action still follows tool permissions.

7.4 Orchestration model

1. Orchestrator creates a task record with objective, scope, risk, budget, and output contract.
2. Context Service creates a reproducible context envelope.
3. Workers run independently where useful and emit signed artifacts.
4. Critic checks evidence, contradictions, assumptions, and policy compliance.
5. Synthesizer composes the result without erasing dissent.
6. Validators enforce schema, citation, safety, and completeness requirements.
7. Approval service intervenes before persistent or external high-impact actions.
8. Workflow commits outputs, logs receipts, and schedules outcome review.

7.5 Disagreement protocol

- Competing conclusions are stored as separate assessments linked to the same question.
- Each assessment exposes evidence, assumptions, priors, model/version, and confidence dimensions.
- The synthesizer identifies the crux: the observation or assumption that would most change the conclusion.
- When evidence cannot resolve the dispute, the output remains plural and defines watch indicators.

8. MCP Tooling and Permission Architecture

8.1 MCP gateway responsibilities

- Tool registry and schema validation.
- Agent and user authentication; scoped authorization.
- Risk classification per operation, not merely per server.
- Credential isolation and secret brokerage.
- Approval checks and signed approval receipts.
- Rate limits, budgets, network restrictions, and sandbox boundaries.
- Idempotency keys, dry-run support, compensation or rollback where available.
- Complete input/output summaries, hashes, status, latency, cost, and related context IDs.

8.2 Permission model

Class	Examples	Default control
Read-only	Read approved files, retrieve memory, query public data.	Pre-authorized by scoped policy; fully logged.
Persistent internal write	Create project artifact, propose memory, update local database.	Validation plus review policy; versioned and reversible.
External communication	Send email, message, publish, submit form.	Explicit approval of recipient, body, attachments, and timing.
State-changing external	Modify calendar, ticket, cloud resource, database, or repository.	Explicit scoped approval; dry run and rollback when possible.
High-impact	Money, employment, legal, medical, identity, access, deletion.	Fresh explicit approval, strong authentication, immutable audit, no broad standing permission.

8.3 Approval receipt

```

approval_id: string
requester: user_id
action_hash: string
scope: exact operation and objects
risk_class: string
issued_at / expires_at: timestamp
conditions: []
result: approved | denied | modified
used_by_trace_id: string
use_count: integer

```

8.4 Tool threat controls

Threat	Control
Prompt injection in retrieved content	Treat content as data; isolate instructions; provenance labels; policy enforcement outside model.
Tool confused-deputy attack	Bind tool authority to user-approved task, agent identity, context, and operation scope.
Credential exfiltration	Secrets never enter model context; use short-lived brokered credentials and output filters.
Destructive retries	Idempotency keys, state checks, transaction boundaries, compensation and manual escalation.
Data overreach	Field/row/path-level restrictions, minimization, and deny-by-default.
Unbounded loops or spend	Budgets, recursion limits, deadlines, circuit breakers, and cancellation.

9. Data, Retrieval, and Knowledge Graph

9.1 Logical stores before technologies

Logical store	Responsibility	Possible implementation
Object registry	Canonical metadata, versions, lifecycle, ownership.	Postgres or SQLite initially.
Event ledger	Append-only state transitions, tool receipts, approvals, provenance.	Postgres event tables or event log.
Document store	Original files, immutable captures, parsed derivatives.	Local object storage, S3-compatible store.
Search index	Lexical and metadata retrieval.	Postgres FTS, OpenSearch, or equivalent.
Vector index	Semantic candidates with model/version lineage.	Qdrant, pgvector, Snowflake Cortex Search.
Knowledge graph	Typed entities and relationships with temporal attributes.	Postgres graph tables first; Neo4j if justified.
Analytical warehouse	Large-scale time series, external datasets, aggregations.	Snowflake when volume and collaboration warrant it.
Archive	Snapshots, exports, retired sources, recovery artifacts.	Versioned cold storage.

9.2 Retrieval pipeline

Intent + entities + time + sensitivity

-> independent candidate channels

-> permission filter

-> temporal validity filter

-> deduplicate and diversify

-> rerank by relevance, reliability, importance, freshness, contradiction coverage

-> context budget allocation

-> retrieval explanation and trace

9.3 Retrieval quality controls

- Golden query sets for projects, people, dates, decisions, and cross-domain questions.
- Chunking by semantic unit with parent-document and claim links; no source lineage loss.
- Embedding model and index version stored with every vector.
- Freshness policy based on claim type, not one generic recency score.
- Source diversity constraints for current-world analysis.
- Negative retrieval tests to prevent sensitive or irrelevant memory leakage.
- Fallback behavior when evidence is insufficient: search broader, expose gap, or abstain.

9.4 Knowledge graph rules

- Edges are typed, sourced, temporally scoped, and confidence-bearing.
- Entity merges are reversible and retain aliases and evidence.
- Causal edges require stronger evidence than simple association.
- Graph traversal is a retrieval aid, not automatic proof.
- Projects, decisions, actions, forecasts, and outcomes are first-class nodes, not just documents.

10. Intelligence, Pattern Recognition, and Forecasting

10.1 Intelligence ladder retained

Level	Artifact	Gate to next level
0	Source	Authenticity, capture, provenance, parse quality.
1	Observation	Faithful extraction and temporal/entity resolution.
2	Signal	Baseline deviation, relevance, novelty, or potential consequence.
3	Pattern	Multiple independent signals, recurrence, scope, counterexamples.
4	Hypothesis	Testability, mechanism, competing explanations, falsifiers.
5	Assessment	Evidence weighting, uncertainty dimensions, alternatives.
6	Forecast	Resolvable proposition, probability, horizon, indicators, base rate.
7	Outcome review	Resolution evidence, scoring, attribution limits, lesson.
8	Model update	Adjusted priors, pattern rules, source weights, workflow or policy.

10.2 Pattern record

- Pattern definition and category: cycle, anomaly, convergence, divergence, cascade, constraint, regime shift, or feedback loop.
- Scope: actors, domains, geography, interval, and valid time.
- Supporting signals and independent source count.
- Baseline or expected comparison.
- Counterexamples and disconfirming evidence.
- Candidate mechanisms and confounders.
- Stability, recurrence, and last observed date.
- Operational implications and watch indicators.

10.3 Forecasting standard

Requirement	Standard
Question	One resolvable proposition or explicitly defined scenario.
Probability	Numeric probability separated from qualitative confidence.
Base rate	Relevant historical or reference class when available.
Evidence	Supporting and counter-signals with time and source lineage.
Assumptions	Explicit, ranked by sensitivity.

Requirement	Standard
Indicators	Observable developments that should update the probability.
Resolution	Objective criteria, source of truth, review date, unresolved rule.
Scoring	Brier or log score for binary/ categorical forecasts; interval coverage where relevant.
Calibration	Reliability curves by domain, horizon, agent/model, and confidence band.

10.4 Decision support contract

- State what is known, inferred, assumed, and unknown.
- Present viable options, expected consequences, reversibility, and key risks.
- Show the evidence that would change the recommendation.
- Distinguish descriptive forecast from normative advice.
- Preserve user values and constraints without manufacturing preferences.
- Create a revisit trigger and outcome review for consequential decisions.

11. Governance, Security, and Threat Model

11.1 Governance authority

Authority object	Purpose
Mind Charter	Values, sovereignty, prohibited behavior, and trust philosophy.
Policy registry	Machine-enforceable permissions, retention, approval, safety, and data rules.
Architecture Decision Records	Explicit technical decisions, alternatives, consequences, and review triggers.
Schema registry	Versioned object and event contracts with migration rules.
Agent and tool registry	Approved versions, owners, permissions, eval status, deployment state.
Risk register	Threats, mitigations, residual risk, owner, and review date.
Change log	Release, migration, configuration, model, prompt, and policy history.

11.2 Data classification

Class	Examples	Minimum control
Public	Public reports, published research, public web data.	Integrity and provenance controls.
Internal	Project plans, private notes without sensitive identity data.	Authenticated access and backups.
Confidential	Personal history, employment, finances, private communications.	Encryption, least privilege, audit, minimization.

Class	Examples	Minimum control
Restricted	Credentials, legal strategy, medical detail, identity documents, highly sensitive relationships.	Strong isolation, explicit use purpose, limited retention, no model exposure unless essential and approved.

11.3 Threat model

Threat class	Required controls
External compromise	Patch discipline, network segmentation, strong authentication, encrypted transport/storage, backups.
Malicious or poisoned sources	Source scoring, immutable capture, cross-source verification, anomaly detection, quarantine.
Prompt injection and model manipulation	Instruction/data separation, allowlisted tools, external policy engine, sandboxing, output validation.
Insider or accidental misuse	Least privilege, approval, audit, separation of duties for critical operations, rollback.
Memory poisoning	Candidate writes, provenance, contradiction checks, review thresholds, version history.
Privacy leakage	Context minimization, sensitivity filters, access tests, redaction, export/deletion controls.
Model/provider risk	Provider policy registry, no secret exposure, routing constraints, local fallback, portability.
Availability failure	Health checks, queues, retry policy, graceful degradation, backup, restore drills.
Silent quality decay	Continuous evals, canaries, drift alerts, release gates, model/prompt version tracking.

11.4 Trust boundary rule

The model is not the security boundary

LLMs may recommend, classify, and explain. Authentication, authorization, approval, policy enforcement, secret handling, transaction control, and immutable audit must be enforced by deterministic services outside the model.

12. Evaluation, Observability, and Learning

12.1 Evaluation hierarchy

Layer	Measures
Schema and unit	Parsing, validation, deterministic transformations, permissions, state transitions.

Layer	Measures
Retrieval	Recall@k, precision@k, ranking quality, temporal correctness, contradiction coverage, sensitive leakage.
Memory	Write accuracy, duplicate rate, contradiction detection, later utility, deletion completeness.
Agent	Contract compliance, evidence fidelity, tool discipline, abstention, cost, latency.
Workflow	End-to-end success, checkpoint/resume, approval correctness, rollback, failure containment.
Intelligence	Signal precision, pattern support, hypothesis testability, forecast calibration.
Human value	Decision usefulness, time saved, error reduction, user corrections, trust calibration.
Security	Policy bypass, injection resistance, privilege escalation, secret leakage, audit completeness.

12.2 Trace model

- Every request receives a trace ID propagated through retrieval, model calls, agents, tools, validations, approvals, writes, and outputs.
- Record model, prompt/template, tool, schema, policy, index, and data versions.
- Capture timing, token/call counts, cost, cache behavior, errors, retries, and cancellation.
- Link final claims and actions back to the objects and tool receipts that support them.
- Protect trace content according to data classification; observability must not become a privacy leak.

12.3 Learning loop

Outcome or correction -> classify failure/success -> identify responsible layer
-> update eval case -> change prompt/model/policy/schema/workflow
-> run regression suite -> canary -> release -> monitor

12.4 Release gates

Change	Minimum gate
Prompt or model	Relevant offline evals, safety tests, cost/latency comparison, canary.
Retrieval/index	Golden query regression, temporal and leakage tests, rollback plan.
Schema	Backward compatibility or migration, fixture tests, versioned readers/writers.
Agent/tool permission	Threat review, policy tests, approval test, audit verification.
Workflow	Success/failure simulations, checkpoint/resume, idempotency and compensation tests.
Security-critical change	Independent review and restore/rollback evidence.

13. Deployment, Operations, and Recovery

13.1 Environment model

Environment	Purpose	Data rule
Development	Fast iteration with synthetic or sanitized data.	No unrestricted production secrets or private memory.
Test / evaluation	Deterministic suites and integration tests.	Curated fixtures and protected evaluation sets.
Staging	Production-like deployment, canaries, migration rehearsal.	Limited replicated data under policy.
Production	Canonical personal Mind services.	Full controls, audit, backup, monitoring, and change gates.

13.2 Operational requirements

- Configuration and secrets separated from code; all changes versioned.
- Queues for asynchronous ingestion and long-running workflows; dead-letter queues with replay policy.
- Checkpoint/resume for multi-step workflows and explicit cancellation semantics.
- Health, readiness, dependency, storage, queue, cost, and quality monitoring.
- Backup schedule, retention, encryption, restore testing, and documented recovery objectives.
- Graceful degradation: local memory and core retrieval remain available when optional external services fail.
- Cost budgets by project, workflow, agent, model, and provider with alerts and circuit breakers.
- Incident classification, containment, evidence preservation, recovery, post-incident review, and eval updates.

13.3 Recovery objectives

Component	Target design expectation
Identity, policies, canonical objects	Highest protection; point-in-time recovery and frequent encrypted backup.
Event ledger and audit	Append-only, tamper-evident, replicated or frequently backed up.
Documents and source captures	Content-addressed storage with checksums and versioning.
Indexes and generated views	Rebuildable from canonical stores; backups optional but recovery procedure required.
Model outputs and transient working state	Retain according to utility, audit, and privacy policy; not all transient state is canonical.

14. Roadmap, Phase Gates, and Acceptance Criteria

The original 24-month roadmap is directionally sound but too time-driven. v2 makes capability and evidence the gate. Calendar ranges are planning estimates, not permission to advance.

Phase	Build focus	Exit evidence
0. Charter and canonical contracts	Mind Charter, domains, ontology, schemas, ADR process, risk model.	No unresolved terminology conflict; schemas validate fixtures; authority hierarchy adopted.
1. Durable core	Object registry, event ledger, local document store, memory candidate pipeline, CLI/API.	Create/read/version/delete/export; provenance retained; backup and restore demonstrated.
2. Retrieval and context	Parsing, lexical/vector/metadata retrieval, context service, trace.	Golden queries meet thresholds; context is reproducible; sensitive leakage tests pass.
3. Tool and workflow control	MCP gateway, policy, approval, workflow engine, filesystem/project/eval tools.	Risk controls, idempotency, audit, cancellation, rollback/compensation tested.
4. Swarm v1	Researcher, Pattern Analyst, Critic, Memory Curator, Synthesizer.	Contract compliance and end-to-end intelligence brief evals pass.
5. Intelligence and forecasting	Signal/pattern/hypothesis/forecast objects, watchlists, outcome review.	Resolvable forecasts, calibration dashboard, pattern evidence rules.
6. Data scale	Warehouse/graph expansion only where workloads justify it.	Measured scale bottleneck resolved; lineage and cost controls preserved.
7. Mind v1	Longitudinal reflection, strategy, project and life decision support.	Demonstrated improvement from outcomes without loss of sovereignty or traceability.

14.1 Build-order corrections

- Build governance, object contracts, and the event ledger before a large agent catalog.
- Build the MCP gateway and permission engine before granting agents broad tools.
- Use a relational core before adding multiple specialized databases; introduce graph or warehouse systems based on measured need.
- Implement evaluation alongside each capability, not as a later phase.
- Create outcome review before claiming forecasting or self-improvement.
- Keep the first vertical slice narrow: one project, one source set, one retrieval path, one bounded workflow, one outcome loop.

15. 10/10 Architecture Audit and Gap Closure

The scores below evaluate the uploaded v1 source set, not the future implementation. v2 raises the design target by converting each weakness into owned work and acceptance evidence.

Area	v1 score	Primary deficit	v2 target remedy
Vision and philosophy	9.0	Strong intent; lacks a machine-enforceable charter and conflict resolution.	Mind Charter, policy registry, sovereignty tests, explicit non-goals.
Conceptual architecture	8.0	Multiple competing layer models.	Faculties + planes + modules + artifacts + domains, each with one purpose.
System decomposition	7.5	Services and ownership not explicit.	Runtime module boundaries, responsibilities, APIs, state and failure contracts.
Context Spine	7.0	Folders and generic metadata, but no context envelope, ownership or duplicate-truth rules.	Canonical domains, generated views, context budget, assembly trace, domain ownership.

Area	v1 score	Primary deficit	v2 target remedy
Object ontology	5.5	Generic type field; missing source/claim/event distinctions and temporal truth.	Universal envelope, type schemas, bitemporal model, lifecycle and lineage.
Memory architecture	7.0	Useful layers; insufficient write policy, contradiction, consolidation, deletion and testing.	Candidate ledger, salience, review, versioning, forgetting, utility metrics.
Agent swarm	6.5	Catalog without executable runtime controls.	Full contracts, orchestration, budgets, checkpoints, validators, disagreement protocol.
MCP and tools	6.0	Risk table and logging, but no enforcement architecture or threat controls.	Gateway, operation-level policy, secrets broker, approvals, idempotency, sandboxing.
Data architecture	6.5	Technology list without canonical stores, lineage or migration rules.	Logical store model, event ledger, version registry, technology ADRs.
Retrieval / RAG	6.5	Pipeline is sound but unmeasured and under-specified.	Independent channels, temporal/security filters, explanation, golden sets, leakage tests.
Knowledge graph	5.5	Basic node list; no temporal, provenance, confidence, identity resolution or graph governance.	Typed sourced edges, reversible merges, temporal graph, causal evidence standards.
Intelligence analysis	7.5	Strong ladder; needs promotion gates, baselines, counterevidence and model updates.	Artifact gates, pattern records, critic role, indicators and post-outcome updates.
Forecasting	7.0	Schema lacks base rates, objective resolution and calibration mechanics.	Resolvable questions, numeric probability, scoring, calibration and review automation.
Governance	6.5	Good principles but not enforceable objects or authorities.	Policy, schema, agent/tool registries, ADRs, risk and change control.
Security	5.0	High-level least privilege and encrypted secrets only.	Threat model, classification, trust boundaries, injection, poisoning, exfiltration and recovery controls.
Evaluation	6.0	Suite categories without datasets, thresholds, release gates or traces.	Layered evals, golden sets, online metrics, version lineage and regression gates.
Observability	4.5	Audit logging mentioned, distributed trace and quality telemetry absent.	End-to-end trace, metrics, cost, versions, claim/action lineage and privacy controls.
Deployment and operations	4.0	No environments, queues, failure recovery, backup, incidents or cost governance.	Operational architecture, recovery objectives, degradation, runbooks and drills.
Roadmap	6.5	Time-based phases with broad exit criteria.	Capability gates, evidence, dependency order, risk reduction and vertical slices.
Implementation readiness	5.5	Major decisions still left to developers during construction.	Schemas, state machines, contracts, examples, tests, ADRs and acceptance criteria.

15.1 Gap closure ownership matrix

Gap	Owning specification	Proof of closure
Vocabulary conflict	Canonical Architecture	Terminology lint; all docs map to canonical model.
Missing schemas	Schema Registry / appendices	Validation against positive and negative fixtures.
Memory safety	Memory Specification	Write, contradiction, deletion, and utility evals.
Agent control	Agent Runtime	Contract tests, budget limits, cancellation, abstention.
Tool safety	MCP and Policy	Permission, approval, injection, secret and idempotency tests.
Retrieval quality	Retrieval Specification	Golden queries, temporal correctness and leakage tests.
Forecast reliability	Intelligence Specification	Resolution automation, scoring and calibration reports.
Operational resilience	Operations Specification	Backup restore, dependency failure and incident drills.
Security completeness	Threat Model	Control verification and residual risk sign-off.
Human sovereignty	Mind Charter + Policy	Correction, veto, export, deletion, approval and override tests.

16. Architecture Decision Records

The following ADRs are established by this reconciliation. Detailed individual ADR files should be created in /operations/architecture/decisions.

ADR	Decision	Consequence
ADR-001	Use faculties for conceptual architecture and planes for technical architecture.	Eliminates competing layer vocabularies.
ADR-002	Treat generated reports and forecast folders as views, not canonical truth stores.	Prevents duplicate truth and stale copies.
ADR-003	Use an object registry plus append-only event ledger as the durable core.	Supports versioning, audit, temporal reconstruction and recovery.
ADR-004	Require candidate validation before canonical memory writes.	Reduces false memories and agent-driven poisoning.
ADR-005	Enforce tool security outside the language model through an MCP gateway and policy engine.	The model cannot grant itself authority.
ADR-006	Separate source reliability, claim confidence, forecast probability, model confidence and retrieval relevance.	Prevents score ambiguity and false precision.

ADR	Decision	Consequence
ADR-007	Start with relational storage and add graph/warehouse/vector specialization by measured workload.	Reduces premature complexity and operational burden.
ADR-008	Make evaluation and observability part of every phase gate.	Quality cannot be postponed until after capability growth.
ADR-009	Store valid time and system time for consequential facts and events.	Supports correction, historical reasoning and decision audit.
ADR-010	Preserve competing hypotheses and dissent through synthesis.	Avoids premature convergence and authority bias.

17. Source Migration and Document Authority

17.1 Canonical source hierarchy

1. Mind Charter - philosophical and sovereignty authority.
2. Canonical Architecture - system model, boundaries, vocabulary, and cross-cutting rules.
3. Security and Governance Specification - enforceable policy and threat controls.
4. Schema Registry - authoritative object and event contracts.
5. Domain specifications - Context, Memory, Agent Runtime, MCP, Data/Retrieval/Graph, Intelligence, Evaluation, Operations.
6. Architecture Decision Records - accepted exceptions and technical decisions.
7. Runbooks and implementation docs - operational procedures and component details.
8. Generated views and reports - useful outputs, never design authority.
9. Archive - superseded sources retained for provenance.

17.2 Migration actions

Action	Result
Move original PDFs and DOCX files to /archive/design/v1/source-set.	Preserves provenance without competing authority.
Place this document at /operations/architecture/00_Canonical_Architecture_v2.	Creates one obvious entry point.
Create a terminology map and lint checklist.	Prevents old layer names from reappearing as architecture.
Revise modular documents to v2 contracts.	Makes detailed specifications consistent with the master.
Create schema files and examples under /operations/schemas.	Turns prose into implementable contracts.
Create ADR-001 through ADR-010.	Makes reconciliation decisions durable and reviewable.
Create a coverage matrix linking requirements to tests.	Ensures no 10/10 target remains merely aspirational.

Appendix A. Canonical Schemas

A.1 Source

source_id, source_type, uri, captured_at, published_at, author, publisher, content_hash, immutable_location, license, reliability_profile, parser_version, security_classification, provenance, related_entities

A.2 Claim

claim_id, proposition, subject, predicate, object, scope, valid_from, valid_to, asserted_by, source_ids, status, confidence, counter_claim_ids, review_date

A.3 Event

event_id, event_type, occurred_start, occurred_end, recorded_at, location, actors, objects, pre_state, post_state, source_ids, causal_claims, confidence

A.4 Memory candidate

candidate_id, proposed_class, content, source_context, salience_factors, sensitivity, duplicate_candidates, contradictions, proposed_retention, review_requirement, producing_trace, disposition

A.5 Forecast

forecast_id, proposition, domain, horizon, probability, base_rate, scenario, supporting_signal_ids, counter_signal_ids, assumptions, indicators, resolution_rule, resolution_source, review_date, status, score, lesson

A.6 Action receipt

action_id, requested_by, approved_by, approval_id, tool_operation, input_hash, idempotency_key, started_at, completed_at, status, effects, output_hash, rollback_available, rollback_status, trace_id

Appendix B. State Machines and Control Flows

B.1 Canonical object lifecycle

candidate -> active -> validated
 candidate -> rejected
 active/validated -> disputed -> validated | superseded
 validated -> superseded -> archived
 any permitted state -> deleted (tombstone + cascade policy)

B.2 Workflow lifecycle

created -> queued -> running -> waiting_for_approval | waiting_for_dependency
 -> running -> validating -> completed
 running -> retrying -> running
 running -> cancelled | failed -> compensated | operator_review

B.3 Forecast lifecycle

draft -> validated -> active -> updating -> active
 active -> due_for_resolution -> resolved | unresolved
 resolved -> scored -> reviewed -> model_update_recorded

B.4 Memory deletion flow

request -> authenticate -> scope discovery -> impact preview -> approval
 -> canonical tombstone -> index deletion -> cache/view regeneration
 -> derived artifact review -> export/update notice -> verification receipt

Appendix C. Initial Backlog

Priority	Work item	Definition of done
P0	Adopt canonical vocabulary and source hierarchy.	v1 sources archived; v2 entry point and terminology map committed.
P0	Create Mind Charter and machine-readable policy registry.	Human-readable charter plus executable tests for approvals, memory and data handling.
P0	Create schema registry for source, observation, claim, entity, event, memory, forecast, decision, action and outcome.	Versioned JSON Schema/Pydantic models, fixtures, migrations and validation CI.
P0	Implement object registry and append-only event ledger.	CRUD/version APIs, bitemporal fields, provenance, audit and backup/restore.
P0	Implement memory candidate pipeline.	Propose, dedupe, contradiction, review, commit, delete and eval paths.
P1	Implement retrieval/context service.	Hybrid candidates, rerank, permission/temporal filters, trace and golden-query evals.
P1	Implement MCP gateway and policy/approval service.	Operation-level enforcement, logs, secrets isolation, idempotency and adversarial tests.
P1	Implement workflow runtime.	Queues, checkpoints, cancellation, retries, compensation, dead-letter and resume.
P1	Build one vertical intelligence slice.	One source domain -> observations -> signals -> pattern -> forecast -> review.
P2	Add agent roles only after contracts and evals exist.	Researcher, Critic, Pattern Analyst, Memory Curator and Synthesizer pass gates.
P2	Add operations and observability stack.	Tracing, metrics, cost, dashboards, alerts, incident and recovery drills.
P3	Add graph and Snowflake based on measured need.	ADR documents bottleneck, options, cost, migration and success measure.

Final Standard

What “real” means in Animus v2

A capability is not considered real because an LLM can demonstrate it once. It is real when its state is represented canonically, its permissions are enforceable, its outputs are traceable, its failures are contained, its quality is measured, its data can be recovered or deleted, and its value survives repeated use.

Animus is not 10/10 today. This architecture defines the evidence required to close every identified gap while preserving the original ambition and human sovereignty.